

Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: **Telekomunikacja (TEL)**

Specjalność: **Sieci teleinformatyczne (TSI)**

PROJEKT ZESPOŁOWY

**Aplikacja do komunikacji głosowej i tekstowej przez sieć
LAN**

Paweł Lipczyk 281330

Michał Gierszon 273182

Konrad Gałka 281277

Kacper Parys 281263

Opiekun projektu:

dr inż. Grzegorz Jaworski

Wrocław 2026

Streszczenie

Celem projektu było stworzenie aplikacji desktopowej umożliwiającej komunikację między użytkownikami w sieci lokalnej (LAN) za pośrednictwem wiadomości tekstowych oraz transmisji głosu. Opracowana aplikacja pozwala na tworzenie pokoju komunikacyjnego na maszynie hosta, do którego mogą dołączać inni użytkownicy znajdujący się w tej samej sieci LAN. Wewnątrz pokoju każdy uczestnik ma możliwość przesyłania wiadomości tekstowych, obrazów oraz plików dowolnego formatu, a także prowadzenia komunikacji głosowej - z możliwością indywidualnej regulacji głośności poszczególnych uczestników lub ich całkowitego wyciszenia. W dalszej części raportu szczegółowo omówiono zastosowane metody implementacyjne wraz z uzasadnieniem podjętych decyzji projektowych.

Abstract

The aim of this project was to develop a desktop application enabling communication between users within a local area network (LAN) through text messaging and voice transmission. The application allows the creation of a communication room on a host machine, which can be joined by other users present within the same LAN. Inside the room, each participant is able to send text messages, images, and files of arbitrary format, as well as engage in real-time voice communication - with support for individual volume control and muting of specific participants. The subsequent sections of this report provide a detailed description of the implementation methods used, along with a justification of the design decisions made throughout the development process.

Spis treści

1	Wstęp	1
1.1	Cel i struktura raportu	1
1.2	Wykorzystanie AI	3
1.3	Opłacalność projektu	3
1.4	Użyteczność społeczna produktu	4
1.5	Plan finansowy projektu	5
1.6	Przegląd rozwiązań dostępnych na rynku	5
2	O szablonie	8
2.1	Definicje użytkownika	9
2.2	Ilustracje	9
2.3	Wzory matematyczne	9
2.4	Cytowania	10
2.5	Tabele	11
2.6	Kody procedur	11
2.7	Akronimy	12
3	Backend	14

Lista skrótów

DRY Don't repeat yourself	12
KISS Keep It Simple, Stupid	12

Rozdział 1

Wstęp

Dynamiczny rozwój narzędzi do komunikacji zdalnej, zapoczątkowany w latach 90. XX wieku wraz z upowszechnieniem się Internetu, doprowadził do powstania szerokiej gamy komunikatorów tekstowych i głosowych. Współczesne rozwiązania, takie jak Microsoft Teams, Discord czy Slack, oferują rozbudowane funkcjonalności, jednak ich architektura opiera się na centralnych serwerach zewnętrznych dostawców - oznacza to, że przesyłane dane opuszczają sieć lokalną i mogą być przetwarzane przez podmioty trzecie.

W środowiskach wymagających zachowania poufności komunikacji, takich jak sieci korporacyjne, laboratoria badawcze czy instalacje przemysłowe, powyższe rozwiązania mogą stanowić istotne zagrożenie dla bezpieczeństwa informacji. Aplikacja VT-LAN powstała jako odpowiedź na tę potrzebę - jest to komunikator tekstowo-głosowy działający wyłącznie w obrębie sieci lokalnej (LAN), w którym pakiety danych nie są nigdy przekazywane poza jej granice. Kwestia bezpieczeństwa znalazła odzwierciedlenie również na poziomie doboru bibliotek - jako warstwę sieciową zastosowano GameNetworkingSockets (GNS), bibliotekę oferującą wbudowane szyfrowanie transmisji oparte na protokole AES-256-GCM oraz uwierzytelnianie połączeń. Takie podejście zapewnia ochronę przesyłanych danych nie tylko poprzez ograniczenie ruchu do sieci lokalnej, lecz także przez szyfrowanie komunikacji na poziomie aplikacji.

Projekt kładzie nacisk na minimalizm i wydajność - aplikacja charakteryzuje się niskim narzutem na zasoby sprzętowe (CPU, GPU, RAM), prostą obsługą oraz skupieniem na realizacji kluczowych funkcji komunikacyjnych bez zbędnych dodatków. Kod źródłowy aplikacji udostępniono na licencji open source, co umożliwia społeczności jego weryfikację, audyt bezpieczeństwa oraz dalszy rozwój.

1.1. Cel i struktura raportu

Celem raportu jest przedstawienie procesu projektowania i implementacji aplikacji VT-LAN. W raporcie omówiono poszczególne moduły składające się na całość systemu, ze szczególnym uwzględnieniem zastosowanych narzędzi, bibliotek oraz przyjętych decyzji projektowych wraz z ich uzasadnieniem.

Zastosowane technologie

Podczas pracy nad aplikacją VT-LAN zespół korzystał z następujących technologii:

1. **Język C++** - główny i jedyny język wykorzystywany do implementacji logiki aplikacji. Odpowiada zarówno za programowanie interfejsu użytkownika, jak i za obsługę komunikacji sieciowej, przechwytywanie oraz odtwarzanie dźwięku. Wybór C++ podyktowany był wymaganiami wydajnościowymi projektu - język ten zapewnia niski

narzut pamięciowy i pełną kontrolę nad zasobami systemowymi, co jest kluczowe przy przetwarzaniu strumieni audio w czasie rzeczywistym.

2. **Microsoft Visual Studio 2022** - zintegrowane środowisko deweloperskie (IDE) wraz z kompilatorem MSVC, wykorzystywane jako główne narzędzie do budowania i debugowania projektu. Dostarcza zaawansowane narzędzia diagnostyczne, w tym debugger z obsługą wielowątkowości oraz profilery wydajności, które były pomocne przy optymalizacji krytycznych fragmentów kodu.
3. **Premake5** - system budowania projektów oparty na skryptach Lua. Umożliwia generowanie plików rozwiązania (.sln) dla Visual Studio z pojedynczego, czytelnego pliku konfiguracyjnego. Zastosowanie Premake5 zamiast bezpośredniej edycji plików projektu upraszcza zarządzanie zależnościami oraz ułatwia konfigurację środowiska członkom zespołu - wszelkie zmiany struktury projektu zawarte są w plikach Lua i automatycznie odzwierciedlane przy kolejnym generowaniu rozwiązania.
4. **GitHub** - platforma do współdzielenia kodu źródłowego oparta na systemie kontroli wersji Git. Służyła jako centralne narzędzie do koordynacji prac zespołu - przeglądania zmian w kodzie, udostępniania aktualizacji oraz śledzenia zadań i błędów za pomocą systemu zgłoszeń. Repozytorium pełni również rolę miejsca dystrybucji projektu - użytkownicy mają możliwość wglądu w kod źródłowy aplikacji, a także pobrania skompilowanej wersji wykonywalnej.
5. **Biblioteki firm trzecich** - projekt korzysta z następujących bibliotek:
 - **SDL3** [1] - wieloplatformowa biblioteka obsługi okna aplikacji, renderowania interfejsu użytkownika oraz systemu audio. W zakresie dźwięku SDL3 dostarcza niskopoziomowe API do otwierania urządzeń wejściowych i wyjściowych, przechwytywania próbek audio z mikrofonu oraz ich odtwarzania przez głośniki. Do miksowania i nakładania strumieni audio aplikacja korzysta z biblioteki **SDL_mixer** [2] - oficjalnego rozszerzenia SDL3 umożliwiającego jednoczesne odtwarzanie wielu kanałów audio z regulacją głośności.
 - **Opus** [3] - otwarty kodek audio o niskim opóźnieniu, zoptymalizowany pod transmisję mowy, ze zmiennym bitrate dostosowującym się do warunków sieci. Bezpośrednio integruje się z buforami próbek pobieranymi przez SDL3, kompresując dane przed wysyłką przez GNS i dekompresując je po stronie odbiorcy. Zastosowanie Opus pozwala znacząco zredukować przepustowość wymaganą do transmisji głosu przy zachowaniu wysokiej jakości dźwięku.
 - **GameNetworkingSockets (GNS)** [4] - biblioteka sieciowa firmy Valve zapewniająca niezawodną, szyfrowaną komunikację opartą na protokole UDP. Oferuje mechanizmy detekcji opóźnień, kontrolę przepływu oraz opcjonalne gwarancje dostarczenia pakietów, co czyni ją dobrze dopasowaną zarówno do komunikacji klient-serwer, jak i peer-to-peer w sieci LAN. Szyfrowanie realizowane jest

przy użyciu algorytmu AES-GCM-256 per pakiet, z wymianą kluczy opartą na Curve25519 [4]. GNS korzysta wewnętrznie z następujących bibliotek:

- **OpenSSL** [5] - biblioteka kryptograficzna zapewniająca szyfrowanie transmisji oraz uwierzytelnianie połączeń,
- **protobuf** [6] - biblioteka Google Protocol Buffers wykorzystywana do serializacji wewnętrznych komunikatów sterujących protokołu GNS.

1.2. Wykorzystanie AI

W trakcie realizacji niniejszej pracy projektowej zespół korzystał z narzędzi generatywnej sztucznej inteligencji zgodnie z wytycznymi Wydziału Informatyki i Telekomunikacji Politechniki Wrocławskiej [7]. Wykorzystywane były następujące narzędzia:

- **Claude (Anthropic, wersja Pro),**
- **ChatGPT (OpenAI, wersja Pro).**

Narzędzia te były stosowane wyłącznie jako pomoc techniczna i redakcyjna, w następujących obszarach:

- **Wspomaganie implementacji** – generowanie fragmentów kodu, odpowiedzi dotyczących struktury rozwiązań oraz debugowanie błędów. Fragmenty kodu powstałe przy udziale narzędzi AI zostały oznaczone stosownymi komentarzami bezpośrednio w raporcie,
- **Wyszukiwanie informacji o bibliotekach i technologiach** – zbieranie informacji na temat dostępnych bibliotek, przyjętych standardów oraz rozwiązań stosowanych w branży (m.in. GameNetworkingSockets, Opus, SDL3),
- **Korekta stylistyczna i językowa tekstu** – poprawa poprawności gramatycznej, spójności stylu oraz dostosowanie języka do wymogów raportu.

Autorzy pracy zachowali pełną odpowiedzialność za merytoryczną poprawność wszystkich zawartych w niej treści. Każda informacja pozyskana przy udziale narzędzi AI była samodzielnie weryfikowana, w szczególności w zakresie poprawności bibliograficznej i technicznej. Narzędzia AI nie były wykorzystywane do generowania całych rozdziałów ani struktury pracy.

1.3. Opłacalność projektu

VT-LAN jest projektem open source udostępnianym nieodpłatnie, którego model finansowy opiera się na zerowym koszcie utrzymania po stronie autorów. Całość infrastruktury działa wyłącznie na maszynach użytkowników w sieci lokalnej - nie istnieje żaden centralny serwer, który wymagałby hostowania, opłacania ani administrowania. Koszt utrzymania projektu po stronie twórców jest zatem zerowy.

Trwałość projektu zapewniana jest przez społeczność - model open source umożliwia każdemu użytkownikowi zgłaszanie poprawek, łatanie usterek oraz rozwijanie nowych funkcjonalności w formie tzw. patchy. Dzięki temu jakość aplikacji może rosnąć bez ponoszenia przez autorów kosztów deweloperskich. Dodatkowym, dobrowolnym źródłem finansowania mogą być dotacje przekazywane przez użytkowników (np. poprzez platformy typu GitHub Sponsors lub Open Collective), co jest powszechnie stosowaną praktyką w projektach open source.

Biorąc pod uwagę powyższe, projekt cechuje się pełną opłacalnością przy zerowym progu wejścia - zarówno dla autorów, jak i dla użytkowników końcowych.

1.4. Użyteczność społeczna produktu

Rosnąca świadomość zagrożeń związanych z prywatnością danych w komunikatorach internetowych czyni VT-LAN odpowiedzią na realną potrzebę społeczną. Popularne platformy komunikacyjne, takie jak Discord, Teams czy Slack, przesyłają dane głosowe i tekstowe przez zewnętrzne serwery swoich dostawców, co oznacza potencjalną ekspozycję tych danych na osoby trzecie.

Trafnym przykładem ilustrującym tę obawę jest kontrowersja wokół Discorda z początku 2026 roku. Platforma ogłosiła wprowadzenie globalnego systemu weryfikacji wieku, wymagającego od użytkowników przesłania skanu dowodu tożsamości lub wykonania skanu twarzy. Decyzja ta wywołała falę krytyki ze strony społeczności i organizacji zajmujących się ochroną prywatności - tym bardziej, że we wrześniu 2025 roku doszło do wycieku zdjęć dokumentów tożsamości nawet 70 000 użytkowników Discorda wskutek naruszenia bezpieczeństwa u zewnętrznego dostawcy usług [8]. Skala reakcji była na tyle duża, że Discord oficjalnie opóźnił wdrożenie systemu do drugiej połowy 2026 roku [9].

Incydent ten uwidocznił fundamentalny problem centralnych komunikatorów: dane użytkowników znajdują się poza ich kontrolą. VT-LAN eliminuje ten problem u podstaw - ruch sieciowy nigdy nie opuszcza sieci lokalnej, a szyfrowanie na poziomie aplikacji (AES-GCM-256) dodatkowo chroni przesyłane treści. Aplikacja jest szczególnie użyteczna w środowiskach wymagających poufności, takich jak sieci korporacyjne, laboratoria, instytucje edukacyjne czy domowe sieci prywatne.

Jako projekt open source VT-LAN wpisuje się ponadto w ideę transparentności oprogramowania - każdy użytkownik może zweryfikować kod źródłowy i upewnić się, że aplikacja działa zgodnie z deklarowanymi zasadami. Repozytorium dostępne publicznie w serwisie GitHub umożliwia pobranie dowolnej wersji projektu wraz z pełnym kodem źródłowym i samodzielne skompilowanie go na własnej maszynie. Dzięki temu użytkownicy nie są uzależnieni od decyzji autorów - w przeciwieństwie do zamkniętych platform, takich jak Discord, żadna przyszła aktualizacja nie może bez wiedzy społeczności wprowadzić mechanizmów weryfikacji tożsamości ani przetwarzania danych biometrycznych.

1.5. Plan finansowy projektu

Ze względu na otwartoźródłowy charakter aplikacji oraz jej architekturę opartą wyłącznie na infrastrukturze użytkownika, VT-LAN nie generuje żadnych stałych kosztów operacyjnych po stronie autorów. Poniżej przedstawiono zestawienie kosztów i potencjalnych przychodów projektu.

Koszty

- **Hosting i infrastruktura serwerowa** - brak. Aplikacja działa wyłącznie na maszynach użytkowników w sieci LAN; autorzy nie ponoszą żadnych kosztów serwerowych.
- **Licencje** - brak. Wszystkie zastosowane biblioteki (SDL3, GameNetworkingSockets, Opus, protobuf) udostępniane są na licencjach open source, niewymagających żadnych opłat.
- **Dystrybucja** - brak. Projekt dystrybuowany jest bezpłatnie za pośrednictwem repozytorium GitHub.
- **Utrzymanie** - pokrywane społecznie w modelu open source poprzez zgłoszenia błędów, patche oraz pull requesty. Autorzy mogą hobbystycznie, w miarę dostępnego czasu, weryfikować i zatwierdzać wkład społeczności lub samodzielnie rozwijać projekt.

Przychody

- **Dobrowolne dotacje użytkowników** – projekt może korzystać z platform umożliwiających dobrowolne wspieranie finansowe twórców open source, takich jak GitHub Sponsors lub Open Collective. Przychody z tego źródła mają charakter nieregularny i zależą od zaangażowania społeczności.

Projekt nie jest nastawiony na generowanie zysku. Jego wartość mierzona jest społecznym zasięgiem, liczbą aktywnych użytkowników oraz wkładem społeczności w jego rozwój.

1.6. Przegląd rozwiązań dostępnych na rynku

Na rynku komunikatorów głosowych i tekstowych funkcjonuje kilka ugruntowanych rozwiązań, które można uznać za punkty odniesienia dla projektu VT-LAN. Poniżej przedstawiono ich charakterystykę wraz z oceną pod kątem prywatności, architektury sieciowej i otwartości kodu.

Discord

Discord [8] to jedna z najpopularniejszych platform komunikacyjnych, oferująca czat tekstowy, głosowy oraz wideo. Posiada rozbudowany ekosystem serwerów tematycznych i integracji z zewnętrznymi usługami. Aplikacja jest bezpłatna w wersji podstawowej, z opcjonalną subskrypcją Discord Nitro.

Z punktu widzenia prywatności Discord jest rozwiązaniem centralnym - cały ruch sieciowy przechodzi przez serwery firmy, co oznacza brak kontroli użytkownika nad przesyłanymi danymi. Kontrowersje wokół próby wprowadzenia przez platformę weryfikacji wieku poprzez skanowanie dokumentu tożsamości i danych biometrycznych, opisane szczegółowo w sekcji 1.4, uwiaryściły systemowe ryzyko wynikające z centralizacji danych [9].

TeamSpeak

TeamSpeak [10] to wieloletnia platforma VoIP skupiona przede wszystkim na komunikacji głosowej, szeroko stosowana w środowiskach graczy oraz w zastosowaniach profesjonalnych. Wyróżnia się niskim opóźnieniem transmisji oraz możliwością uruchomienia własnego serwera, co daje administratorowi pełną kontrolę nad infrastrukturą. Wersja podstawowa jest bezpłatna, natomiast utrzymanie własnego serwera wiąże się z kosztami hostingu. Kod źródłowy pozostaje zamknięty.

Mumble

Mumble [11] jest otwartoźródłowym komunikatorem głosowym o niskim opóźnieniu, działającym w modelu klient-serwer. Serwer (Murmur) może być uruchomiony w sieci lokalnej, co eliminuje konieczność przesyłania danych przez serwery zewnętrzne. Aplikacja jest w pełni bezpłatna i dostępna na licencji open source. Słabą stroną Mumble jest brak natywnej obsługi przesyłania plików.

Microsoft Teams

Microsoft Teams to korporacyjna platforma komunikacyjna zintegrowana z ekosystemem Microsoft 365. Oferuje czat tekstowy, połączenia głosowe i wideo, a także współdzielenie plików i zaawansowane narzędzia pracy zespołowej. Jest rozwiązaniem wyłącznie chmurowym - dane przetwarzane są na serwerach Microsoft, co w środowiskach o podwyższonych wymaganiach bezpieczeństwa może stanowić istotne ograniczenie. Produkt jest płatny w ramach subskrypcji Microsoft 365.

Podsumowanie i pozycja VT-LAN

VT-LAN zajmuje unikalną niszę wśród powyższych rozwiązań. Jest jedynym komunikatorem zaprojektowanym z myślą o działaniu wyłącznie w sieci LAN, łączącym czat tekstowy, transmisję głosową i przesyłanie plików w jednej lekkiej aplikacji open source, niewymagającej żadnej infrastruktury zewnętrznej ani kont użytkowników.

Tabela 1.1. Porównanie komunikatorów dostępnych na rynku

Cecha	Discord	TeamSpeak	Mumble	MS Teams	VT-LAN
Czat tekstowy	Tak	Ograniczony	Ograniczony	Tak	Tak
Głos (VoIP)	Tak	Tak	Tak	Tak	Tak
Przesyłanie plików	Tak	Nie	Nie	Tak	Tak
Tylko LAN	Nie	Nie	Możliwe	Nie	Tak
Open source	Nie	Nie	Tak	Nie	Tak
Własny serwer	Nie	Tak	Tak	Nie	Tak
Bezpłatny	Częściowo	Częściowo	Tak	Nie	Tak
Narzut zasobów	Wysoki	Niski	Niski	Wysoki	Niski

Rozdział 2

O szablonie

W rozdziale tym znajduje się kilka wskazówek, które mogą Ci się przydać podczas redagowania pracy oraz przykłady użycia pakietów systemu Latex, które są dołączone do tego szablonu. Szablon ten bazuje na klasycznej klasie `report`. Najważniejszym składnikiem tego szablonu jest plik `settings.tex`. Oto kilka szczegółów:

1. Wszystkie pliki pracy powinny być zapisane w formacie utf-8.
2. Posługujemy się czcionkami z rodziny `newtxtext` oraz `newtxmath`. Są to nowoczesne wersje fontów w stylu Times. Głównym fontem dokumentu jest czcionka szeryfowa. Strona tytułowa jest wyświetlana wariantem bezszeryfowym.
3. Do wyświetlania bibliografii posługujemy się pakietem `biblatex`, a do przetwarzania bibliografii programem `biber`. Jest to nowoczesna wersją starego programu `bibtex`. Przed użyciem tego szablonu dowiedz się, jak skonfigurować środowisko, które używasz do składania plików Latex'a, tak aby używało ono `biber`'a. Jeśli masz z tym problem, spróbuj z linii poleceń uruchomić polecenie `biber plik`, gdzie `plik` jest nazwą głównego pliku projektu bez rozszerzenia. Jeśli korzystasz ze środowiska [Overleaf](#), to ono samo powinno wykryć stosowanie bibera.
4. Jeśli korzystasz z lokalnych środowisk do składania plików Latex'a (np. TeXnicCenter), to pierwsza kompilacja z użyciem tego szablonu może zająć trochę czasu, gdyż środowisko może doinstalowywać potrzebne pakiety.
5. Szablon importuje pakiet `listings` do wyświetlania "listingów" programów oraz pakiety `algorithm` i `algpseudocode` do wyświetlania pseudokodów.
6. Szablon importuje pakiet `acro` do operowania synonimami. Jeśli nie stosujesz synonimów, to zakomentuj dwie linie głównego pliku (są one opisane komentarzami w głównej stronie szablonu). Swoje synonimy wpisz do pliku o nazwie `synonimy.tex`.
7. Szablon korzysta z pakietu `cleveref` do bardziej elastycznego stosowania referencji wewnątrz dokumentu. Sprawdź możliwości tego pakietu: na przykład sprawdź co da skorzystanie z polecenia `\cref{. .}` zamiast standardowego `\ref{. .}` do odwoływania się do rysunków lub tabel.

Podziel swoją pracę na szereg mniejszych plików (na przykład: `Rozdzial01.tex`, `Rozdzial02.tex`, itd). Rozdziały te dołączaj do głównego pliku pracy za pomocą poleceń **`include`** lub **`input`** (include forsuje wstawienie polecenia nowej strony). Rozdziały umieszczaj w podkatalogu `chapters`.

2.1. Definicje użytkownika

Jeśli w pracy kilka razy posługujesz się tym samym fragmentem tekstu, to warto zdefiniować swoje własne polecenia LaTeX'a. Oto przykład wykorzystania definicji takich jak `\newcommand{\NN}{\mathbb{N}}` z głównego pliku tego szablonu. Oto przykład:

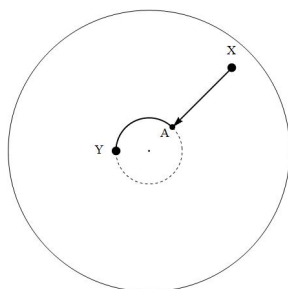
Symbolem $\mathbb{R}$ będziemy oznaczać zbiór liczb rzeczywistych, zaś symbolem $\mathbb{N}$ będziemy oznaczać zbiór liczb naturalnych. Zwróćmy uwagę na to, że liczbę 0 zaliczamy do zbioru liczb naturalnych, czyli $\mathbb{N} = \{0, 1, 2, \dots\}$.

Stosowanie takich definicji znacznie ujednolica tekst pracy oraz eliminuje szereg błędów (co jest chyba oczywiste dla każdego informatyka).

2.2. Ilustracje

Za pomocą polecenia `\includegraphics` można wstawiać obrazy do dokumentu LaTeX'a (potrzebna jest do tego celu, dołączona do szablonu, klasa `graphicx`). Obrazy umieszczaj w podkatalogu `figures`.

Dokładny opis parametrów tego polecenia możesz znaleźć na stronie https://latexref.xyz/_005cincludegraphics.html.



Bardziej elastyczny sposób polega na otoczeniu grafiki konstrukcją `picture` (patrz Rys. Rys. 2.1). Poniżej znajduje się Rys. 2.1.

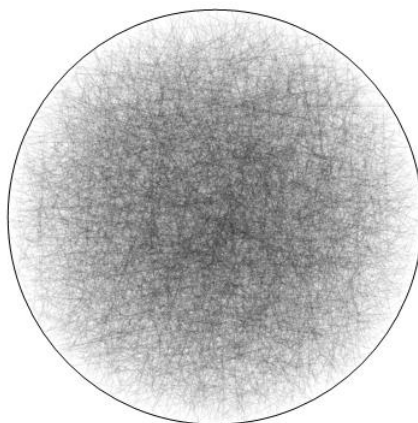
Szablon pracy dyplomowej wykorzystuje pakiet `cleveref`. Służy on do elastycznego operowania referencjami wewnątrz dokumentu. Na przykład, używając komendy `cref` (zamiast standardowej `\ref{fig:Lines}`), nie musisz samemu pisać skrótu Rys.

Jeśli odkomentujesz polecenie `\listoffigures` w głównym pliku, to rysunek ten znajdzie się na wygenerowanej liście rysunków.

2.3. Wzory matematyczne

Wzory matematyczne w obrębie paragrafu formatuje się za pomocą polecenia `$ \dots $`, np. $f(x) = \sin(x^2)$. Za pomocą polecenia `\[ \dots \]` formatujesz równania wycelowane bez numerowania, np.

$$\sin^2(\alpha) + \cos^2(\alpha) = 1 ,$$



Rysunek 2.1. Symulacje 10000 losowych odcinków w kółku jednostkowym. Rysunek został samodzielnie wygenerowany.

a za pomocą konstrukcji `\begin{equation} ... \end{equation}` formatujesz równania numerowane, np.

$$\exp(I \cdot t) = \cos(t) + I \cdot \sin(t) \quad (2.1)$$

Równań numerowanych używaj **tylko wtedy**, gdy będziesz się do nich odwoływać w dalszej części tekstu za pomocą polecenia `\ref{...}`.

Jeśli w pracy znajdują się jakieś twierdzenia, lematy, to skorzystaj z konstrukcji `theorem`, `lemma`, `corollary` oraz `proof`, zdefiniowanych w pakiecie `amsthm`, zaimportowanym do tego szablonu. Oto przykład:

Twierdzenie 2.1 (Euklides). *Istnieje nieskończenie wiele liczb pierwszych.*

Dowód. Załóżmy, że $p_1, \dots, p_n$ jest listą wszystkich liczb pierwszych. Niech

$$q = (p_1 \cdot \dots \cdot p_n) + 1.$$

Wtedy $q > 1$, istnieć więc musi liczba pierwsza p taka, że $p|q$

□

2.4. Cytowania

W tym szablonie korzystamy z biblioteki `biber` do formatowania cytatów. Jest to nowocześniejsza wersja standardowej (ale obecnie już nie rozwijanej) biblioteki `bibtex`. Oto przykład użycia:

Książka [12] jest poświęcona podstawowym algorytmom. Autor Knuth omawia w niej większość klasycznych algorytmów. Z punktu widzenia tej pracy ważny jest algorytm znajdujący się w Knuth [13, s. 249].

Prace [14], [15] zawierają zwięzłe wprowadzenia do problematyki *routingu geograficznego*.

Wpisy opisujące źródła bibliograficzne najlepiej jest przechowywać w oddzielnym pliku o formacie `bibtex`. Do szablonu dołączony jest przykładowy plik `bibliography.bib`.

Znajdują się w nim przykłady podstawowych typów tzw. wpisów. Porządnie sformatowane wpisy bibliograficzne możesz znaleźć, na przykład, na stronach [dblp](#) oraz w serwisie [Google Scholar](#).

Biber, używany z pakietem biblatex, zapewnia wiele predefiniowanych stylów formatowania cytatów i bibliografii, z typowymi stylami takimi jak `numeric`, `alphabetic`, `authoryear`, `authortitle` i `verbatim`, wraz z opcjami sortowania, takimi jak `nty` (name, title, year) lub `nyt` (name, year, title). Style te są wybierane za pomocą opcji `style=` w poleceniu

```
\usepackage[backend=biber, style=...]{biblatex}
```

w preambule LaTeX'a. Stosowany w pracy styl formatowania ustal wspólnie z opiekunem pracy. W pliku `main.tex` zaproponowano styl numeryczny bez opcji sortowania, co oznacza, że bibliografia będzie wyświetlana w kolejności pojawienia się cytowanej pozycji w tekście pracy.

Uwaga

Cytowania w LaTeX'u możesz samodzielnie obsługiwać bez pomocy pliku w formacie bibtex. Jednak samodzielne, poprawne sformatowanie bibliografii jest dość żmudne. Pamiętaj również o tym, że każda pozycja występująca w spisie literatury musi być cytowana w tekście pracy.

2.5. Tabele

Tabele wstawiasz w sposób standardowy. Po odkomentowaniu polecenia `\listoftables` z głównego pliku tego szablonu do pliku zostanie dołączona lista tabel. Do tej tabeli możesz się odwoływać za pomocą polecenia `\cref{tab:example}` (Tab. 2.1).

Tabela 2.1. Przykładowa tabela

Kategoria	Parametr	Wynik
Test A	Niskie obciążenie	15 ms
Test B	Wysokie obciążenie	120 ms

2.6. Kody procedur

Do tego szablonu dołączony został pakiet `listings`. Ułatwia on wstawianie kodów źródłowych do dokumentu. W szablonie, w pliku `ustawienia.tex`, są ustawione domyślne parametry wyświetlania dla kodów.

Oto przykład jego użycia:

```
1  /*
2   * Algorytm Euklidesa.
3   * Zwraca gcd(a, b).
4   */
5  int gcd(int a, int b)
6  {
```

```

7   while (b != 0) {
8       int r = a % b;
9       a = b;
10      b = r;
11  }
12  return a;
13 }

```

Listing 2.1. Algorytm Euklidesa w języku C

Oto inny przykład, kod implementacji algorytmu QuickSort w języku Haskell (listing 2.2) wykorzystujący, dla zwiększenia czytelności kodu, mechanizm *list comprehension*:

```

1  -- QuickSort: czas O(n^2); średni czas O(n log(n))
2  qs [] = []
3  qs (x:xs) = qs [t | t <- xs, t < x] ++
4               [x] ++
5               qs [t | t <- xs, t >= x]

```

Listing 2.2. Algorytm QuickSort w języku Haskell

Do wstawiania pseudokodów do pracy zalecamy korzystanie z pakietów `algorithm` oraz `algpseudocode` dołączonych do tego szablonu. Oto przykład pseudokodu zapisanego w języku tych pakietów (algorytm 1):

Algorytm 1 Filtrowanie

```

1: Initialize:  $S \leftarrow \emptyset$ 
2: for  $v \in V$  do
3:   if condition( $v$ ) then
4:      $S \leftarrow S \cup \{v\}$ 
5:   end if
6: end for
7: return  $S$ 

```

▷ Dodaj jeśli spełnia warunek

2.7. Akronimy

Szablon ten korzysta z pakietu `acro`. Plik `acronims.tex` jest przykładem pliku konfiguracyjnego. Oto przykład użycia:

Podczas budowania oprogramowania dużą wagę przykładaliśmy do przestrzegania dwóch zasad: zasady Don't repeat yourself (DRY) oraz zasady Keep It Simple, Stupid (KISS)

Za drugim razem, gdy użyjesz synonimów, pojawią się już tylko skrócone nazwy, na przykład:

Stosowanie zasady DRY polega unikaniu pisania kilkakrotnie podobnych fragmentów kodu. Stosowanie tej zasady znacznie ułatwia modyfikację kodów. Z kolei stosowanie zasady KISS zmusza programistę do systematycznego zastanawiania się nad jakością swojego kodu.

Jeśli nie masz potrzeby korzystania z akronimów, to zakomentuj dwie linie w pliku `main.tex` (linie 59 oraz 73).

Rozdział 3

Backend

Spis literatury

- [1] SDL Community. *Simple DirectMedia Layer (SDL)*. 2026. URL: <https://github.com/libsdl-org/SDL>.
- [2] SDL Community. *SDL_mixer – An audio mixer library for SDL*. 2026. URL: https://github.com/libsdl-org/SDL_mixer.
- [3] Jean-Marc Valin, Koen Vos i Timothy Terriberry. *Definition of the Opus Audio Codec*. RFC 6716. Internet Engineering Task Force (IETF), wrzesień 2012. URL: <https://www.rfc-editor.org/rfc/rfc6716>.
- [4] Valve Corporation. *GameNetworkingSockets – Reliable & unreliable messages over UDP*. 2026. URL: <https://github.com/ValveSoftware/GameNetworkingSockets>.
- [5] OpenSSL Software Foundation. *OpenSSL – Cryptography and SSL/TLS Toolkit*. 2026. URL: <https://www.openssl.org>.
- [6] Google LLC. *Protocol Buffers – Google’s data interchange format*. 2026. URL: <https://github.com/protocolbuffers/protobuf>.
- [7] PWi WIT. *Wytyczne dot. narzędzi GenAI*. 2026. URL: https://wit.pwr.edu.pl/wydzial/jakosc_ksztalcenia/wytyczne-dot-narzedzigenai.
- [8] Rindala Alajaji i Samantha Baldwin. *Discord Voluntarily Pushes Mandatory Age Verification Despite Recent Data Breach*. Luty 2026. URL: <https://www.eff.org/deeplinks/2026/02/discord-voluntarily-pushes-mandatory-age-verification-despite-recent-data-breach>.
- [9] Mauricio B. Holguin. *Discord is now delaying its global age verification plans after huge backlash*. 2026. URL: <https://alternativeto.net/news/2026/2/discord-is-now-delaying-its-global-age-verification-plans-after-huge-backlash>.
- [10] TeamSpeak Systems GmbH. *TeamSpeak – Voice Communication for Online Gaming*. 2025. URL: <https://www.teamspeak.com> (dostęp 2026-06-04).
- [11] Mumble Project. *Mumble – Open Source, Low-Latency, High Quality Voice Chat*. 2025. URL: <https://github.com/mumble-voip/mumble> (dostęp 2026-06-04).
- [12] Donald E. Knuth. *The art of computer programming. Volume 1: Fundamental Algorithms*. 3rd. T. 1. The art of computer programming. Reading, Massachusetts: Addison-Wesley, 1997.
- [13] Donald E. Knuth. *The art of computer programming. Volume 2: Seminumerical Algorithms*. 3rd. T. 2. The art of computer programming. Reading, Massachusetts: Addison-Wesley, 1997.
- [14] Fabian Kuhn, Roger Wattenhofer i Aaron Zollinger. „An algorithmic approach to geographic routing in ad hoc and sensor networks”. W: *IEEE/ACM Trans. Netw.* 16.1 (2008), s. 51–62.

- [15] L. Popa i in. „Balancing Traffic Load in Wireless Networks with Curveball Routing”.
W: *Mobihoc'07*. 2007.